

fn sample_geometric_exp_fast

Michael Shoemate (@Shoeboxam)

October 6, 2026

Proves soundness of `fn sample_geometric_exp_fast` in `mod.rs` at commit `0be3ab3e6` (outdated¹). This proof is an adaptation of subsection 5.2 of [CKS20].

1 Hoare Triple

Precondition

Compiler-verified

- Argument `x` is of type `RBig`, a bignum rational.

Caller-verified

- $x \geq 0$

Pseudocode

```
1 def sample_geometric_exp_fast(x: RBig) -> int:
2     if x == 0:
3         return 0
4
5     s, t = x.into_parts()
6
7     while True:
8         u = sample_uniform_ubig_below(t) #
9         d = sample_bernoulli_exp(Rational(u, t)) #
10        if d:
11            break
12
13        v = sample_geometric_exp_slow(1) #
14        z = u + t * v
15        return z // s
```

Postcondition

Theorem 1.1. For any setting of the input parameter `x` such that the given preconditions hold, `sample_geometric_exp_fast` either returns `Err(e)` due to a lack of system entropy, or `Ok(out)`, where `out` is distributed as $\text{Geometric}(1 - \exp(-x))$.

Assume the preconditions are met.

Lemma 1.2. There exist a non-negative integer s and positive integer t such that $x = s/t$. If $x > 0$, then $s \geq 1$.

¹See new changes with `git diff 0be3ab3e6..4e331c2 rust/src/traits/samplers/cks20/mod.rs`

Proof. Since x is of type **Rbig** and $x \geq 0$, it can be written as $x = s/t$ for some non-negative integer s and positive integer t . This is why **RBig.into_parts** is infallible. If $x > 0$, then its numerator is positive, so $s \geq 1$. $\square$

Lemma 1.3. **sample_geometric_exp_fast** only returns **Err(e)** when there is a lack of system entropy.

Proof. If $x = 0$, the function returns **Ok(0)** immediately, so it cannot return **Err(e)**. Assume $x > 0$. By Lemma 1.2, there exist integers $s \geq 1$ and $t > 0$ with $x = s/t$. Since t is a positive integer, the preconditions on **sample_uniform_ubig_below** are met. Any returned value u therefore lies in $\{0, 1, \dots, t-1\}$, so $u/t \geq 0$, and the preconditions on **sample_bernoulli_exp** are also met. The input on line 13 is the non-negative rational 1, so the preconditions on **sample_geometric_exp_slow** are met as well. By their definitions, each of these fallible calls can only return an error due to a lack of system entropy. These are the only fallible calls in **sample_geometric_exp_fast**, so the function only returns **Err(e)** when there is a lack of system entropy. $\square$

We now establish some lemmas that will be useful in proving the distribution of **out**.

Lemma 1.4. Suppose $x > 0$. Conditioned on the loop on lines 8–9 terminating without returning **Err(e)**, the value u at loop exit is a realization of a random variable U_{acc} supported on $\{0, 1, \dots, t-1\}$, where

$$P[U_{\text{acc}} = u] = \frac{1 - e^{-1/t}}{1 - e^{-1}} e^{-u/t}.$$

Proof. By Lemma 1.2, $t > 0$, so in each loop iteration the call on line 8 samples a random variable $U \sim \text{Uniform}(0, t)$ supported on $\{0, 1, \dots, t-1\}$. Conditioned on a realization u of U , the call on line 9 samples $D \sim \text{Bernoulli}(\exp(-u/t))$ by the specification of **sample_bernoulli_exp**.

Each iteration uses fresh randomness, and the loop stops at the first iteration for which $D = \top$. Therefore the value returned by the loop has the same distribution as U conditioned on $D = \top$. For any $u \in \{0, 1, \dots, t-1\}$,

$$\begin{aligned} P[U_{\text{acc}} = u] &= P[U = u \mid D = \top] \\ &= \frac{P[U = u]P[D = \top \mid U = u]}{P[D = \top]} && \text{by Bayes' theorem} \\ &= \frac{(1/t)e^{-u/t}}{\frac{1}{t} \sum_{k=0}^{t-1} e^{-k/t}} \\ &= \frac{(1/t)e^{-u/t}}{\frac{1}{t} \frac{1 - e^{-1}}{1 - e^{-1/t}}} \\ &= \frac{1 - e^{-1/t}}{1 - e^{-1}} e^{-u/t}. \end{aligned}$$

$\square$

Lemma 1.5. Suppose $x > 0$ and **sample_geometric_exp_fast** returns **Ok(out)**. Then the value z is a realization of a random variable $Z \sim \text{Geometric}(1 - \exp(-1/t))$.

Proof. By Lemma 1.4, the value u exiting the loop is distributed as U_{acc} . The call on line 13 returns a value v distributed as $V \sim \text{Geometric}(1 - \exp(-1))$ by the specification of **sample_geometric_exp_slow**. Since line 13 is executed after the loop and uses fresh randomness, U_{acc} and V are independent.

For any z , define $u_z := z \bmod t$ and $v_z := \lfloor z/t \rfloor$, so that $z = u_z + t \cdot v_z$. Then

$$\begin{aligned}
P[Z = z] &= P[U_{\text{acc}} = u_z, V = v_z] && \text{since } z = u_z + t \cdot v_z \\
&= P[U_{\text{acc}} = u_z]P[V = v_z] && \text{by independence} \\
&= \frac{1 - e^{-1/t}}{1 - e^{-1}} e^{-u_z/t} \cdot (1 - e^{-1}) e^{-v_z} && \text{by Lemma 1.4} \\
&= (1 - e^{-1/t}) e^{-(u_z/t + v_z)} \\
&= (1 - e^{-1/t}) e^{-z/t}.
\end{aligned}$$

Therefore $Z \sim \text{Geometric}(1 - \exp(-1/t))$. $\square$

Lemma 1.6. [CKS20] Fix $p \in (0, 1]$. Let G be a $\text{Geometric}(1 - p)$ random variable, and $n \geq 1$ be an integer. Then $\lfloor G/n \rfloor$ is a $\text{Geometric}(1 - q)$ random variable with $q = p^n$.

Proof.

$$\begin{aligned}
P[\lfloor G/n \rfloor = k] &= P[nk \leq G < (k+1)n] && \text{any } G \text{ in the interval maps to } k \\
&= \sum_{l=nk}^{(k+1)n-1} (1-p)p^l \\
&= (1-p^n)p^{nk} \\
&= (1-q)q^k
\end{aligned}$$

$\square$

Lemma 1.7. If the outcome of `sample_geometric_exp_fast` is `Ok(out)`, then `out` is distributed as $\text{Geometric}(1 - \exp(-x))$.

Proof. If $x = 0$, the function returns `Ok(0)` immediately. Since $\text{Geometric}(1 - \exp(-0)) = \text{Geometric}(0)$ is the point mass at 0, the claim holds.

Assume $x > 0$. By Lemma 1.2, there exist integers $s \geq 1$ and $t > 0$ with $x = s/t$. By Lemma 1.5, `z` is a realization of $Z \sim \text{Geometric}(1 - \exp(-1/t))$. Applying Lemma 1.6 with $p = \exp(-1/t)$ and $n = s$, we conclude that $\lfloor Z/s \rfloor$ is distributed as $\text{Geometric}(1 - \exp(-s/t))$. Finally, `out` = $\lfloor Z/s \rfloor$ and $s/t = x$, so `out` is distributed as $\text{Geometric}(1 - \exp(-x))$. $\square$

Proof. Theorem 1.1 holds by 1.3 and 1.7. $\square$

References

[CKS20] Clément L. Canonne, Gautam Kamath, and Thomas Steinke. The discrete gaussian for differential privacy. *CoRR*, abs/2004.00010, 2020.